

FINAL YEAR PROJECT • 2025 — 2026

ForsaDrive

A Tunisian-context carpooling platform — Web, Mobile, and a unified backend

Presented by

Youssef BEN ABID & Anas YOUNES

Supervised by

Mr. Khalil SELMI (ATOMIC IT)

Ms. Ines BEN NASR (Academic)

Outline of the defense

01

Project context & problem

Why mobility in Tunisia is a real problem

02

Existing solutions

What people use today and where it falls short

03

Proposed solution & methodology

ForsaDrive concept and the Scrum process

04

Conception

Architecture, use cases, class diagram

05

Realization — sprint by sprint

From foundations to community features

06

Tests & demo

What works and a live walk-through

07

Conclusion & future work

What we delivered and what comes next

Mobility in Tunisia — a real, daily problem

Public transport is irregular. Private cars travel half empty. Existing apps are not built for the local market.

Public transport

Overcrowded buses, irregular schedules, weak coverage of secondary cities.

Informal carpooling

Facebook groups, no identity verification, no payment trace, no rating.

International apps

BlaBlaCar/inDrive/Bolt — payment models and language not adapted to Tunisia.

PROBLEM STATEMENT

Tunisia lacks a secure, affordable digital ride-sharing solution adapted to local payment habits, available on web and mobile, and capable of building genuine trust between drivers and passengers.

ATOMIC IT — host company

- ▶ IT engineering services: web and mobile development, software consulting.
- ▶ Modern stack: Linux/Windows, .NET, Java, Android/iOS, PHP.
- ▶ Located in Kelibia, Nabeul. Director: Mr. Khalil SELMI.
- ▶ Flat structure — short feedback loops between developers and management.
- ▶ Internship inside the engineering team, mentored by senior developers.
- ▶ Agile Scrum applied: daily stand-ups, weekly reviews with the supervisor.

AT A GLANCE

Activity	Web & mobile development
Director	Mr. Khalil SELMI
Location	Kelibia, Nabeul
Phone	+216 55 343 224
Email	contact@atomicitpro.com
Methodology	Agile / Scrum

What people use today

Criterion	Facebook	inDrive	Bolt	BlaBlaCar	ForsaDrive
Identity verification	—	Partial	✓	✓	✓
In-app payment	—	—	✓	✓	✓
Long-distance rides	✓	—	—	✓	✓
Rating system	—	✓	✓	✓	✓
Adapted to local context	✓	Partial	Partial	—	✓
Student discount (50%)	—	—	—	—	✓
Wallet-based balance	—	—	—	—	✓

No existing option combines local fit with structured trust mechanisms. ForsaDrive sits exactly in that gap.

ForsaDrive — what we built

An integrated ecosystem: web app, mobile app, and a shared backend, covering the full lifecycle from registration to post-trip rating.

Local fit

50% prepayment + cash settlement,
French/English/Arabic with RTL, Tunisian-
recognized university domains.

Trust

Identity & student verification, ratings on both
sides, complaint workflow, reliability score.

Smart features

Compatibility score, driver analytics, ride
boosting, social feed, in-app chat, AI-assisted
helpdesk.

3

coordinated apps

4

functional sprints

20+

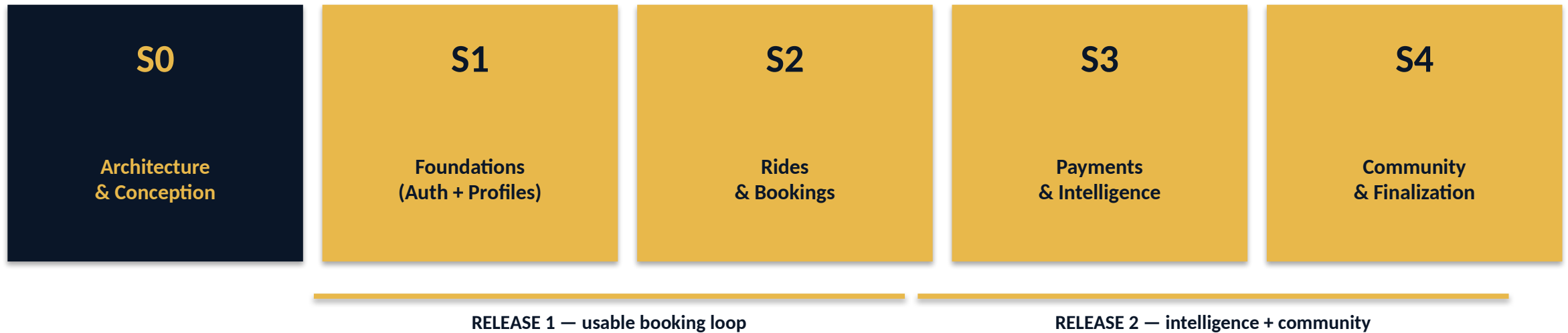
domain entities

3

supported languages

Scrum — five iterations grouped into two releases

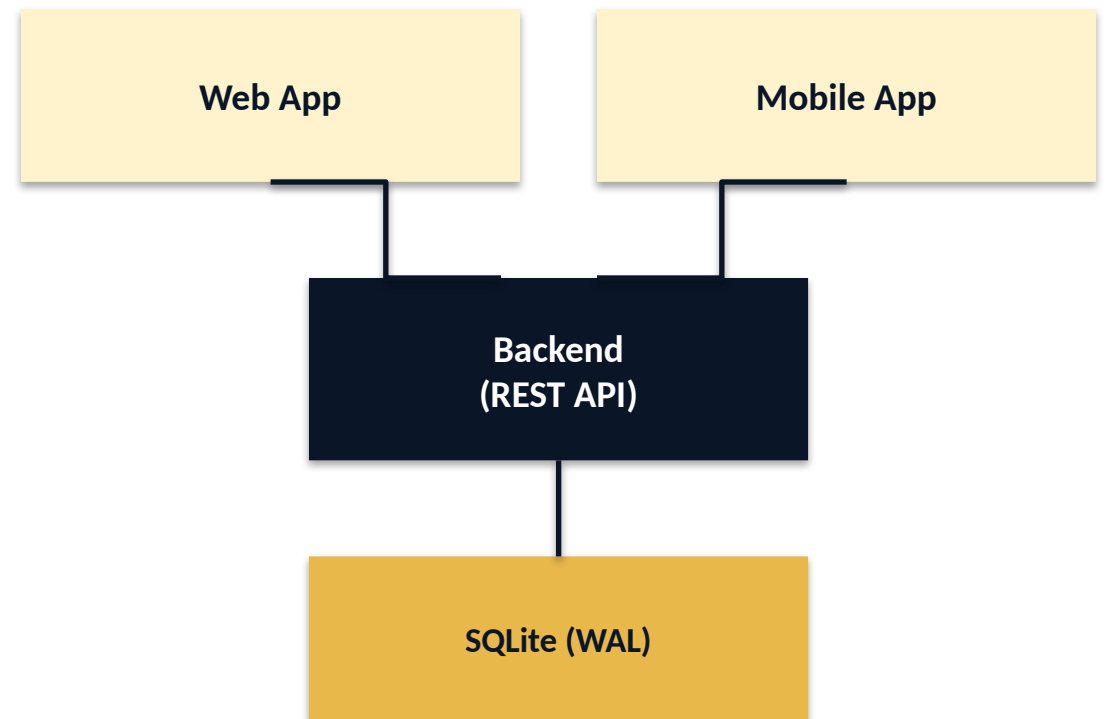
Sprint 0 prepared the ground. Four functional sprints then delivered two coherent releases of the product.



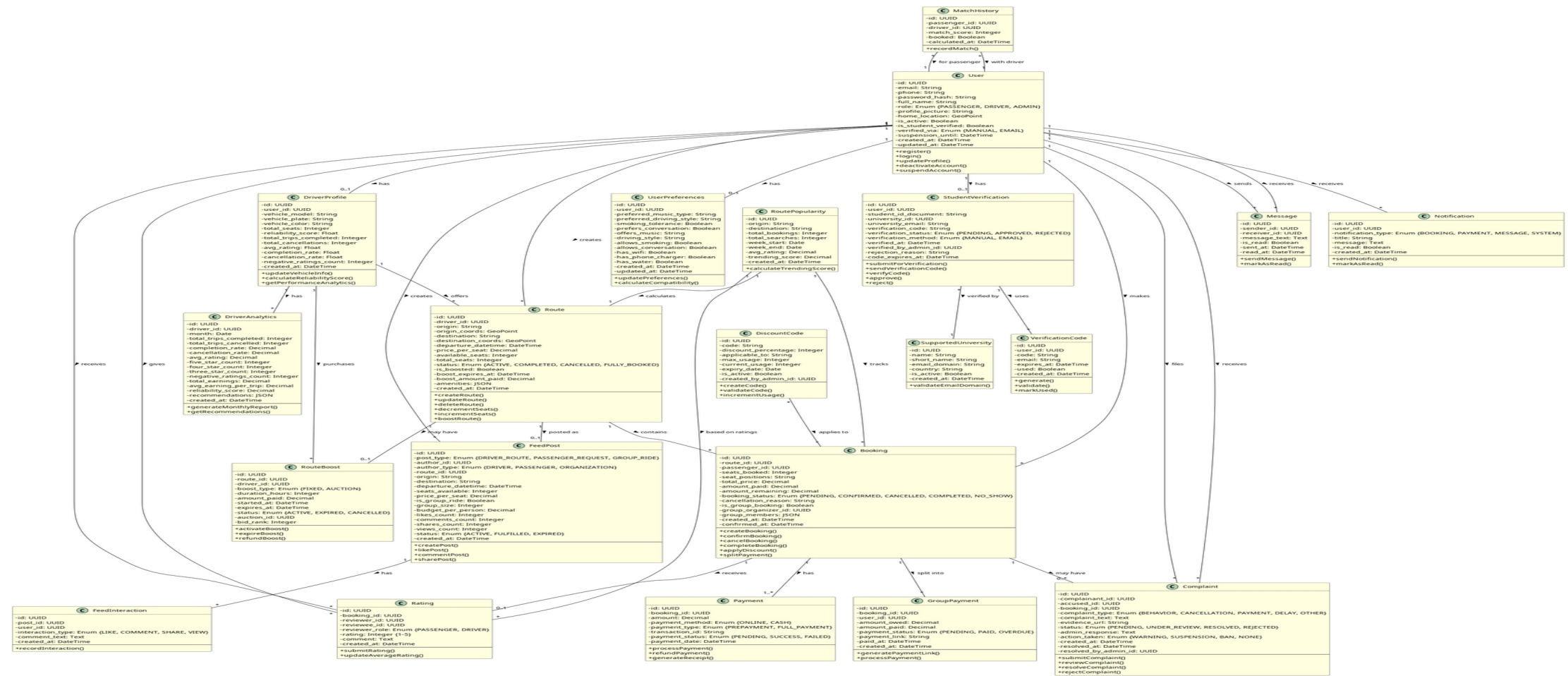
- ▶ Daily stand-ups inside the team — weekly reviews with the supervising engineer.
- ▶ Each sprint produced a usable increment, reviewed and demonstrated.
- ▶ Trello board kept the backlog visible — story points on a Fibonacci scale.

Architecture — client / server, three tiers

- ▶ Two clients (web + mobile) consume one REST API.
- ▶ Backend in PHP, exposed under /api/, secured with bearer tokens.
- ▶ Three tiers: presentation, application, data.
- ▶ External services (payment, notifications, file storage) only reached through the backend — no client secrets.
- ▶ HTTPS in production. Passwords with bcrypt. Audit log on sensitive operations.

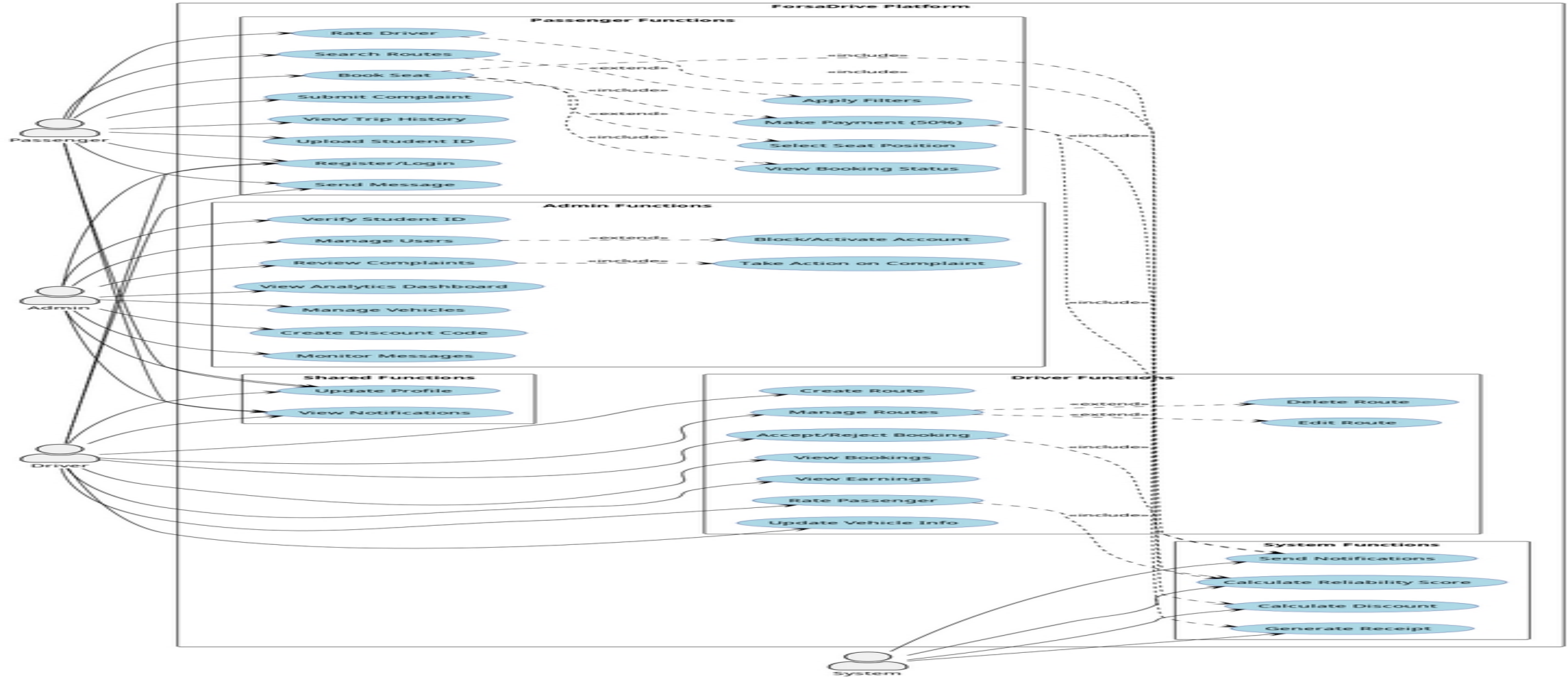


Class diagram – 20+ entities, 5 logical groups



Users & Profiles • Trips & Bookings • Payments & Ratings • Verification & Discounts • Communication & Moderation

Global use case — four actors



Passenger • Driver • Administrator • System (automated)

SPRINT 1

Foundations

MISSION

Account creation, authentication, profile, student verification, driver application & admin approval.

Sprint backlog (extract)

#	User story	Pts
US1.1	Register an account	5
US1.2	Login with email + password	3
US1.3	View / update profile	3
US1.4	Student verification via university email OTP	8
US1.5	Apply to become a driver (license + vehicle)	8
US1.6	Admin reviews driver applications	5
US1.7	Admin manages recognized university domains	3

Deliverables

- ▶ Self-service OTP flow (10-min code, 5-attempt cap, 30s cooldown).
- ▶ Driver application creates DriverProfile + Vehicle on approval.
- ▶ Audit log entry on every verification event.

SPRINT 2

Rides & Bookings

MISSION

Driver publishes a ride. Passenger searches, filters, books with 50% prepayment.

Sprint backlog (extract)

#	User story	Pts
US2.1	Publish a ride (origin, destination, date, price, seats)	5
US2.2	Modify or cancel a ride	3
US2.3	Search rides by origin / destination / date	5
US2.4	Filter by price, departure time, driver rating	5
US2.5	View ride detail and driver profile	3
US2.6	Book seats with 50% prepayment	8
US2.7	Group booking for friends	5
US2.8	Driver accepts / rejects booking requests	5

Deliverables

- ▶ Booking persisted only if prepayment succeeds — no orphan bookings.
- ▶ Search response indexed on origin, destination, departure_date.
- ▶ Activity diagram covered cash collection at end of trip.

SPRINT 3

Payments & Intelligence

MISSION

Wallet, promo codes, ride boosting, driver analytics, compatibility score.

Sprint backlog (extract)

#	User story	Pts
US3.1	View wallet balance and history	5
US3.2	Top up wallet	3
US3.3	Auto-apply 50% student discount on booking	3
US3.4	Apply organizational promo code	3
US3.5	Boost a ride to top of search/feed	5
US3.6	Driver dashboard with reliability + revenue + ratings	8
US3.7	Compatibility score badge per ride	5
US3.8	Recompute reliability after every completed trip	3

Deliverables

- ▶ Price pipeline: $\text{base} \times \text{seats} \rightarrow \text{student } 50\% \rightarrow \text{promo } \% \rightarrow 50\% \text{ prepayment}$.
- ▶ Reliability score: $\text{rating} \times \text{completion} \times (1 - \text{cancellation})$.
- ▶ Bug caught at sprint review: promo applied before student discount — fixed.

SPRINT 4

Community & Finalization

MISSION

Chat, ratings, complaints, social feed, helpdesk, multilingual interface, admin panel.

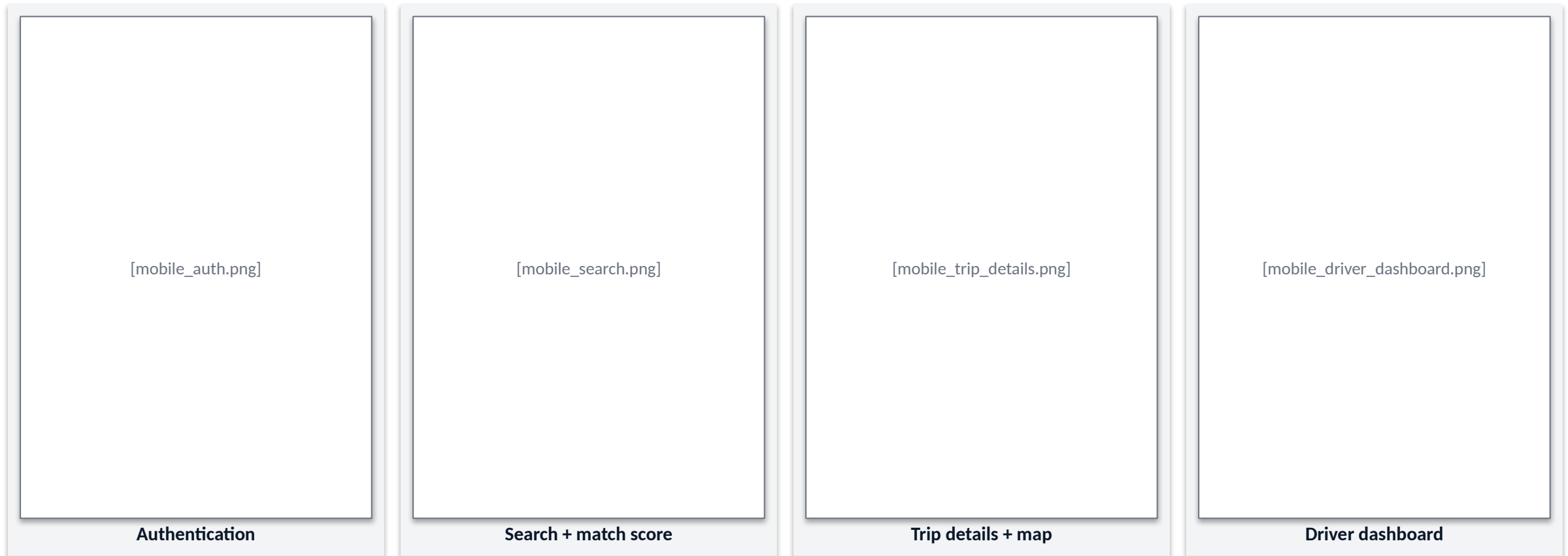
Sprint backlog (extract)

#	User story	Pts
US4.1	Real-time in-app chat (poll-based)	5
US4.2	Push notifications (FCM)	3
US4.3	Social feed (posts, likes, comments)	8
US4.4	Rate the other party after a trip	3
US4.5	File a complaint after a trip	5
US4.6	AI-assisted HelpDesk with bot escalation	5
US4.7	Switch interface FR / EN / AR (RTL)	5
US4.8	Admin panel — 7 tabs	8

Deliverables

- ▶ Chat polled every 3 s — sufficient for current scale, no WebSocket dep.
- ▶ HelpDesk bot covers 14 FAQ categories, escalates to human agent.
- ▶ Arabic flips the layout automatically through Flutter localizations.

Selected screens



Web counterparts cover the same workflows for desktop users and the administration panel.

What only the mobile makes possible

Push notifications

Firebase Cloud Messaging delivers booking, message, and ride alerts even when the app is in the background.

Interactive map

OpenStreetMap tiles + OSRM routing draw the driving line between origin and destination on each ride detail screen.

Real-time chat

Conversations linked to bookings — bubble UI, date dividers, read receipts. 3-second polling, no WebSocket dependency.

Multilingual + RTL

FR / EN / AR with automatic right-to-left layout flipping for Arabic, no manual override in any widget.

Technology stack

Web frontend

PHP 8 (server-rendered)

Routing, sessions, page rendering.

HTML5 + Bootstrap 5

Responsive layout, consistent components.

Vanilla JavaScript

Form validation, AJAX, admin tab switching.

Backend / data

PHP REST API

Front-controller under /api/, JSON responses.

PDO + bearer tokens

Parameterized queries, 32-byte hex tokens.

SQLite (WAL mode)

Shared file between web and mobile.

Mobile

Flutter (Dart)

Single codebase, native rendering on Android & iOS.

Provider + go_router

State management + declarative navigation.

FCM, flutter_map, l10n

Push, OSM map, FR/EN/AR with RTL.

Testing — continuous, not deferred

Strategy

- ▶ Unit tests on the price pipeline, prepayment, booking state machine.
- ▶ Postman collections replay end-to-end scenarios after each backend change.
- ▶ Manual cross-device tests on three Android phones.
- ▶ Weekly demo with the supervising engineer caught usability issues.

Results

Scenario	Result
Registration + student OTP	✓ Passed
Token-based session	✓ Passed
Driver application + approval	✓ Passed
Publish / cancel a ride	✓ Passed
Book with 50% prepayment	✓ Passed
Student discount + promo code	✓ Passed
Wallet top-up + history	✓ Passed
Ratings + complaints	✓ Passed
Real-time chat + push (FCM)	✓ Passed
Multilingual + RTL	✓ Passed

LIVE DEMO

ForsaDrive in action

Walk-through: registration → student verification → search → book → driver acceptance → cash collection.

Limitations & roadmap

What's missing today

- ▶ Wallet uses manual top-ups — no real payment gateway integrated yet.
- ▶ SQLite is suited for development, not high-traffic production.
- ▶ Mobile tested only on Android — iOS build not verified.
- ▶ No forgot-password email flow yet.
- ▶ Performance under concurrent load was not measured.

Next steps

- ▶ Migrate to PostgreSQL or MySQL for production.
- ▶ Integrate a Tunisian payment gateway (Konnect or Paymee).
- ▶ Deploy backend behind HTTPS on a VPS, automated backups.
- ▶ Publish the Flutter app on the Google Play Store.
- ▶ Add 2FA for driver and admin accounts + offline cache on mobile.

QUESTIONS & DISCUSSION

Thank you for your attention.

We are now ready for your questions.

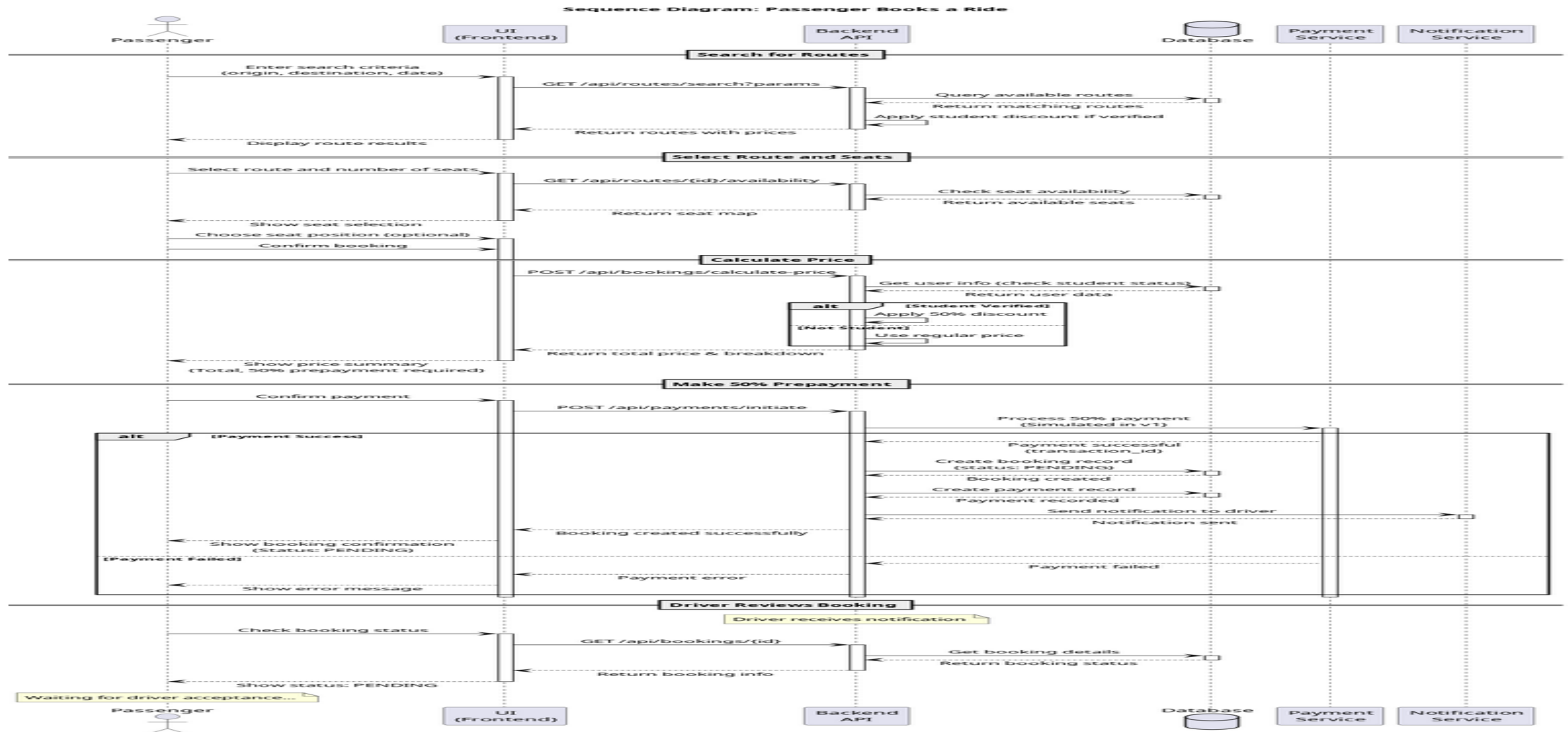
Youssef BEN ABID & Anas YOUNES • ForsaDrive • 2025 — 2026

BACKUP MATERIAL

Annexes — for questions

The following slides are not part of the main presentation. They are kept available in case the jury asks for deeper detail.

Sequence — passenger books a ride



Prepayment is deducted from the wallet BEFORE the booking is persisted — guarantees no orphan bookings.

Sequence — student OTP verification



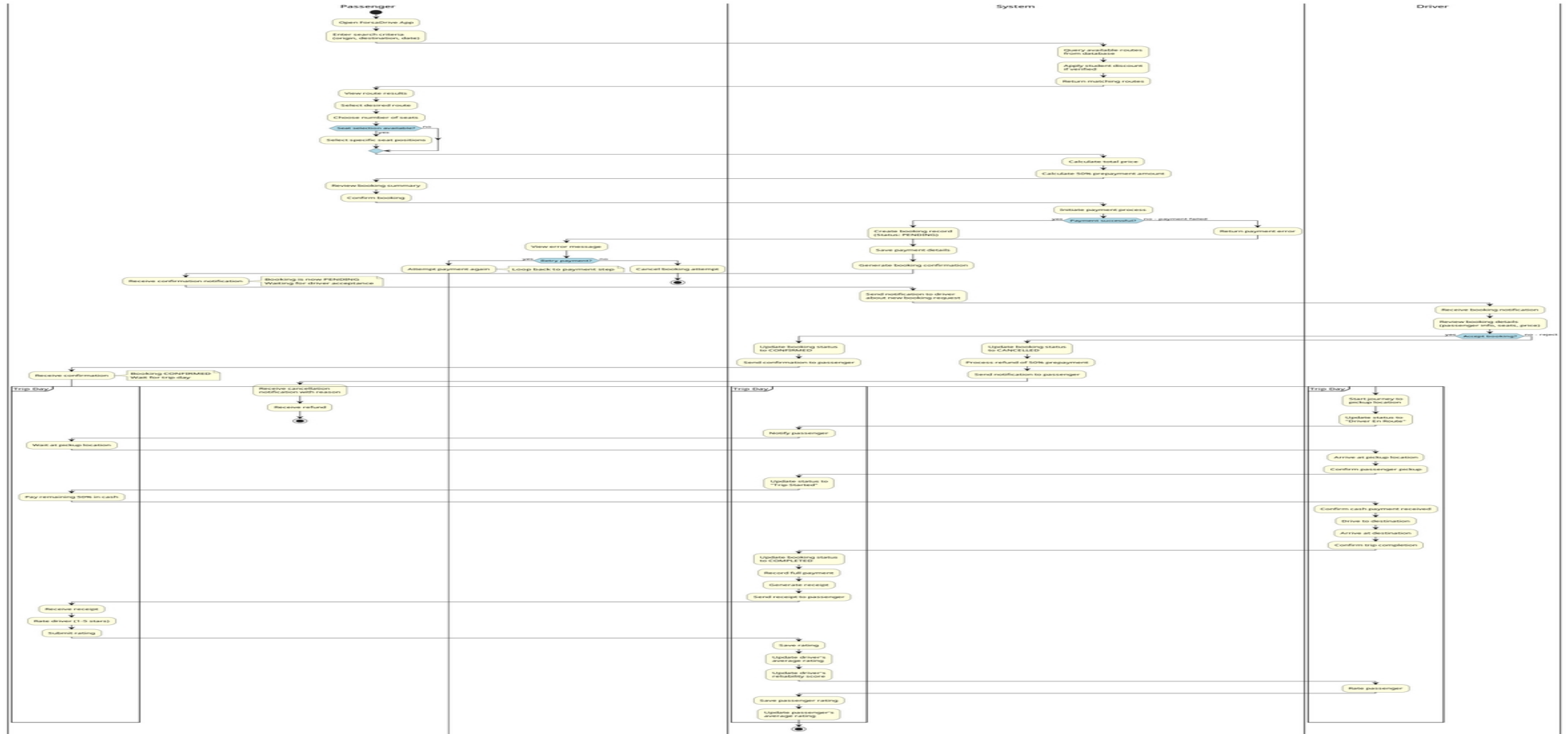
Domain checked against student_domains • 6-digit OTP, 10-min expiry, 5-attempt cap, 30s cooldown between sends.

Sequence — driver application & admin approval

[image: ForsaDrive_Sequence_DriverApplication.png]

On approval: DriverProfile + Vehicle are created and the user role is switched to DRIVER atomically.

Activity diagram — end-to-end booking journey



Three swimlanes: passenger / system / driver. Decision points: retry payment, accept/refuse, depart, arrive, cash collection.

Price computation pipeline

Order matters. Reviewed and validated with the supervising engineer.

1	base price	<code>price_per_seat × seats_booked</code>
2	student discount	<code>if is_student_verified: price × 0.5</code>
3	promo code	<code>if org code valid: price × (1 – pct/100)</code>
4	prepayment	<code>prepayment = price × 0.5</code>
5	persist	<code>store all 5 values on the booking row</code>

Bug caught at Sprint 3 review: steps 2 & 3 were inverted. Fixed same evening, locked by a unit test.

Core relational schema (extract)

Table	Columns
users	id, username, email, password, is_driver, is_student, balance, score
driver_profiles	id, user_id*, license_number, avg_rating, total_trips, reliability
vehicles	id, user_id*, type, make, model, plate_number, seats, has_ac, has_wifi
rides	id, driver_id*, vehicle_id*, from_location, to_location, departure_time, price, available_seats, status
bookings	id, ride_id*, passenger_id*, seats, paid_amount, status
payments	id, user_id*, amount, type, description, ref_id
ratings	id, ride_id*, from_user_id*, to_user_id*, score, comment
student_verifications	id, user_id*, university_email, status, verified_at
organizations	id, name, type, contact_email, email_domain, discount_percent, discount_code, status
complaints	id, from_user_id*, against_user_id*, type, text, status, admin_note
messages	id, conversation_id*, sender_id*, body, is_read, sent_at

* foreign key. ~20 tables total in the live schema. Indexes on origin, destination, departure_time, foreign keys.

REST API surface (extract)

Method	Path	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Returns 32-byte bearer token
GET	/api/rides	Search with from/to/date/filters
POST	/api/rides	Driver publishes a ride
DELETE	/api/rides/{id}	Driver cancels
POST	/api/bookings	Passenger books — 50% prepayment
POST	/api/bookings/group	Group booking
POST	/api/bookings/validate-promo	Org promo code lookup
POST	/api/bookings/requests/{id}	Driver accepts/rejects
POST	/api/student/send-otp	Send OTP to university email
POST	/api/student/verify-otp	Verify OTP, mark verified
GET	/api/admin/organizations	List org accounts (admin)
POST	/api/admin/organizations/{id}/review	Approve / reject org
GET	/api/feed	Social feed (posts + interactions)

All protected endpoints require the Authorization: Bearer <token> header.

Security posture

Concern	Mitigation
Password storage	bcrypt via PHP password_hash
Session	Random 32-byte hex bearer token, 30-day TTL
Transport	HTTPS in production (XAMPP HTTP for dev)
SQL injection	PDO with parameterized queries everywhere
XSS	Server-rendered output is escaped (htmlspecialchars)
CSRF (web admin)	Same-origin admin panel + session-based auth
Admin separation	Admin login blocked on the mobile API (web-only)
File uploads	Mime + size validation; served from /Src under XAMPP
Audit trail	Sensitive ops (verification, suspension) logged server-side
Rate limiting	OTP send has a 30s per-account cooldown